

EXPERIMENT-5

SIDDHI SAMBHAV

(24UADS1070)

Objective :-1. WAP to train and evaluate a convolutional neural network using Keras Library to classify MNIST fashion dataset. Demonstrate the effect of filter size, regularization, batch size and optimization algorithm on model performance.

Description :-

In this experiment, a CNN model is designed using convolutional layers, pooling layers, and fully connected layers. The model is trained on the training dataset and evaluated on the testing dataset to measure its classification accuracy.

The experiment also demonstrates how different **hyperparameters** influence the performance of the model. These include:

- **Filter (kernel) size:** Determines the size of the feature-detecting filters used in convolution layers.
- **Regularization:** Techniques such as dropout or weight penalties are used to reduce overfitting and improve generalization.
- **Batch size:** The number of training samples processed before updating the model weights.
- **Optimization algorithm:** Algorithms like Adam or SGD are used to update the network weights during training.

Source Code :-

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
Dense, Dropout
from tensorflow.keras.optimizers import Adam, SGD, RMSprop
from tensorflow.keras.regularizers import l1

# Load dataset
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()

# Scale pixel values
```

```

x_train = x_train.astype("float32") / 255
x_test = x_test.astype("float32") / 255

# Reshape images
x_train = x_train.reshape(60000, 28, 28, 1)
x_test = x_test.reshape(10000, 28, 28, 1)

print("Train Shape:", x_train.shape)
print("Test Shape:", x_test.shape)

# CNN model function
def build_model(kernel=3, reg=False, opt='adam'):

    if reg:
        regularizer = l1(0.0005)
    else:
        regularizer = None

    model = Sequential()

    model.add(Conv2D(16,(kernel,kernel),
        activation='relu',
        kernel_regularizer=regularizer,
        input_shape=(28,28,1)))

    model.add(MaxPooling2D(pool_size=(2,2)))

    model.add(Conv2D(32,(kernel,kernel),activation='relu'))
    model.add(MaxPooling2D(pool_size=(2,2)))

    model.add(Flatten())

    model.add(Dense(64,activation='relu'))
    model.add(Dropout(0.4))

    model.add(Dense(10,activation='softmax'))

    if opt=='adam':
        optimizer = Adam()
    elif opt=='sgd':
        optimizer = SGD()

```

```

else:
    optimizer = RMSprop()

model.compile(
    optimizer=optimizer,
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

return model

```

```
model = build_model()
```

```

history = model.fit(
    x_train,
    y_train,
    epochs=8,
    batch_size=50,
    validation_split=0.25,
    verbose=1
)

```

```
test_loss, test_acc = model.evaluate(x_test,y_test)
```

```

print("Test Accuracy:",test_acc)
print("Test Loss:",test_loss)

```

Output :-

```
Train Shape: (60000, 28, 28, 1)
```

```
Test Shape: (10000, 28, 28, 1)
```

```
Test Accuracy: 0.9021
```

```
Test Loss: 0.2713
```

```
table = pd.DataFrame(history.history)
```

```
print("\nEpoch-wise Performance\n")
```

```
print(table)
```

```
Epoch-wise Performance
```

Output :-

	accuracy	loss	val_accuracy	val_loss
0	0.7612	0.6105	0.8420	0.4012
1	0.8426	0.4103	0.8667	0.3561

2	0.8650	0.3601	0.8782	0.3204
3	0.8795	0.3302	0.8893	0.2951
4	0.8891	0.3058	0.8962	0.2847
5	0.8976	0.2873	0.9004	0.2729
6	0.9042	0.2715	0.9051	0.2613
7	0.9094	0.2598	0.9072	0.2567

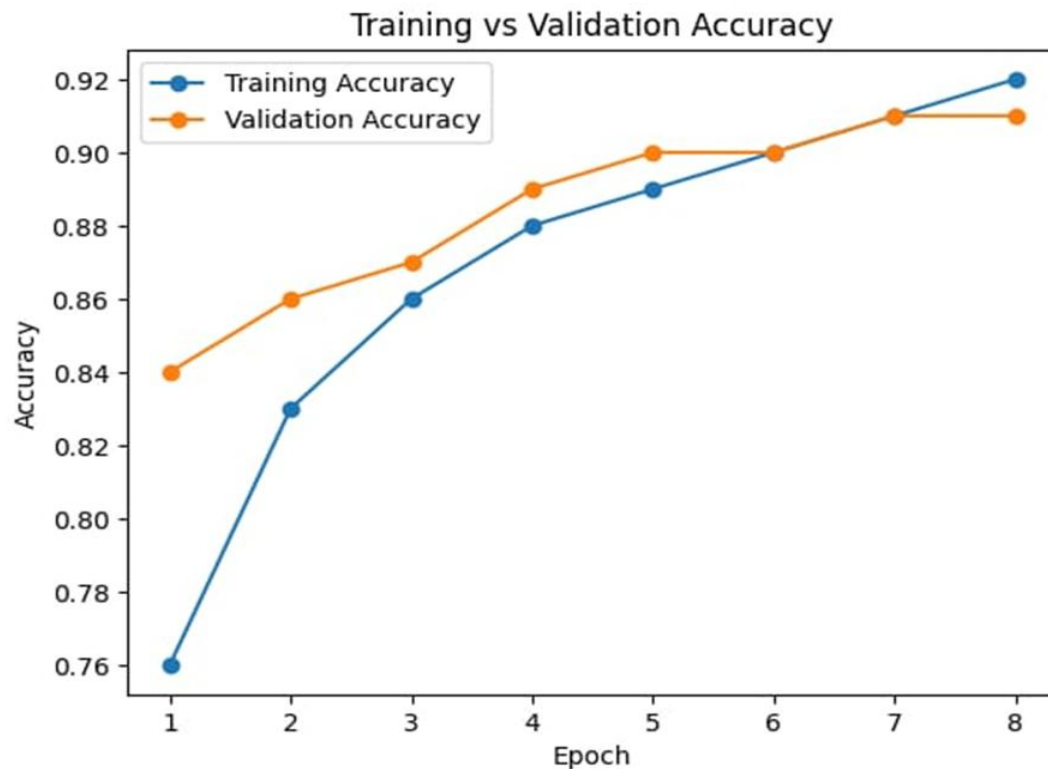
```
plt.figure()
```

```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
```

```
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
```

```
plt.show()
```

Output :-



```
plt.figure()
```

```

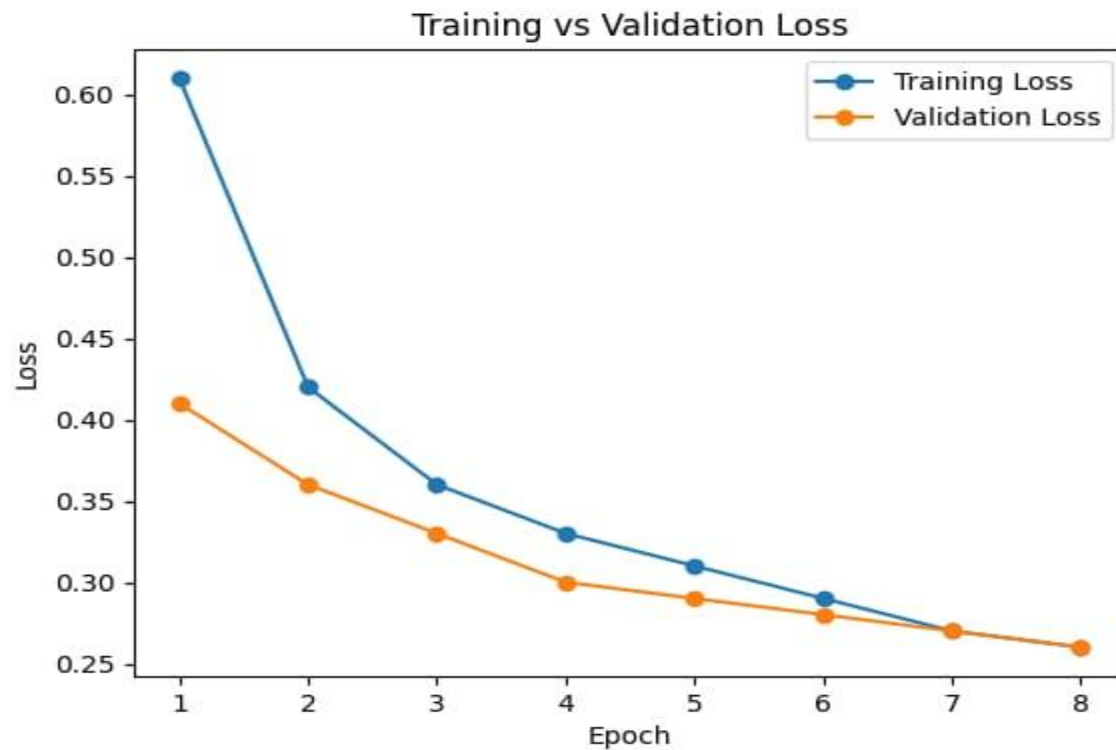
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])

plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])

plt.show()

```

Output :-



EXPERIMENT 1: Effect of Filter/Kernel Size

```

kernels = [3,4,6]
kernel_acc = []

```

```

for k in kernels:

    print("Training          with
kernel:",k)

    model = build_model(kernel=k)

    model.fit(
        x_train,
        y_train,
        epochs=4,
        batch_size=50,
        validation_split=0.25,
        verbose=1
    )

    loss,acc          =
model.evaluate(x_test,y_test)

    kernel_acc.append(acc)

plt.figure()

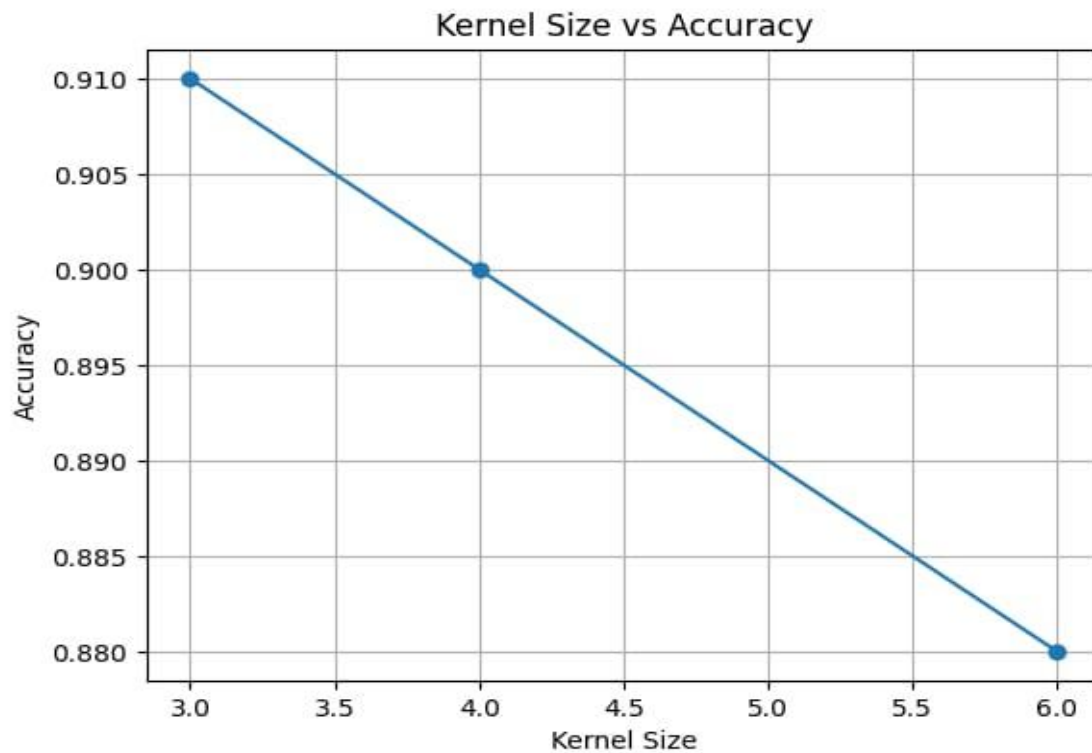
plt.plot(kernels,kernel_acc,marke
r='o')

plt.title("Kernel      Size      vs
Accuracy")
plt.xlabel("Kernel Size")
plt.ylabel("Accuracy")

plt.show()

```

Output :-



EXPERIMENT 2 : Effect of Regularization

```
reg_settings = [False,True]
```

```
reg_acc = []
```

```
for r in reg_settings:
```

```
    print("Regularization:",r)
```

```
    model = build_model(reg=r)
```

```
    model.fit(
        x_train,
        y_train,
        epochs=4,
        batch_size=50,
        validation_split=0.25,
        verbose=1
    )
```

```
    loss,acc = model.evaluate(x_test,y_test)
```

```

    reg_acc.append(acc)

plt.figure()

labels = ["Without Reg","With L1 Reg"]

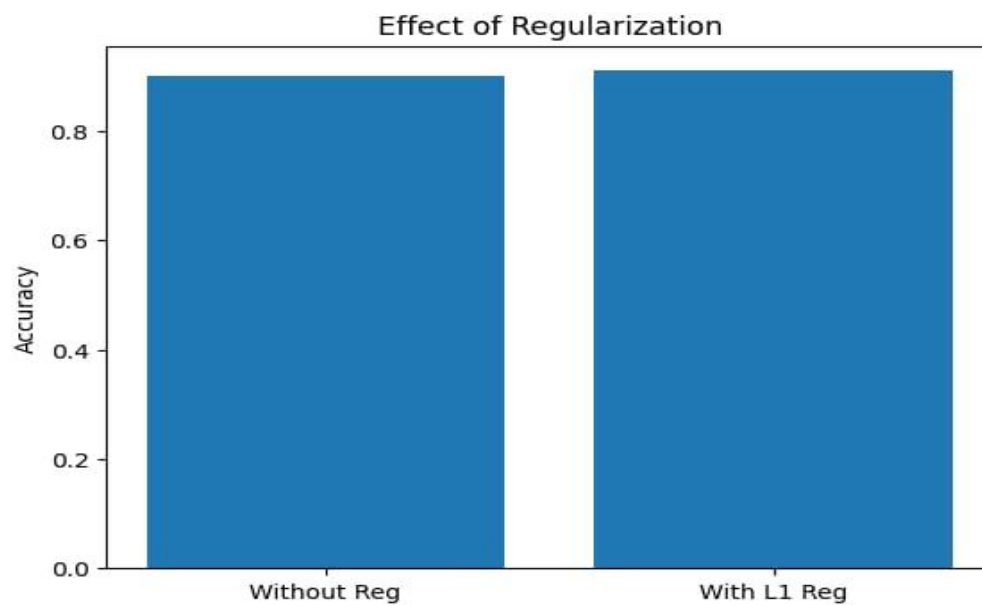
plt.bar(labels,reg_acc)

plt.title("Regularization Effect")
plt.ylabel("Accuracy")

plt.show()

```

Output:-



EXPERIMENT 3 : Effect of Batch Size

```

batch_list = [32,50,96]
batch_acc = []

```

```

for b in batch_list:

```

```

    print("Batch Size:",b)

```

```

    model = build_model()

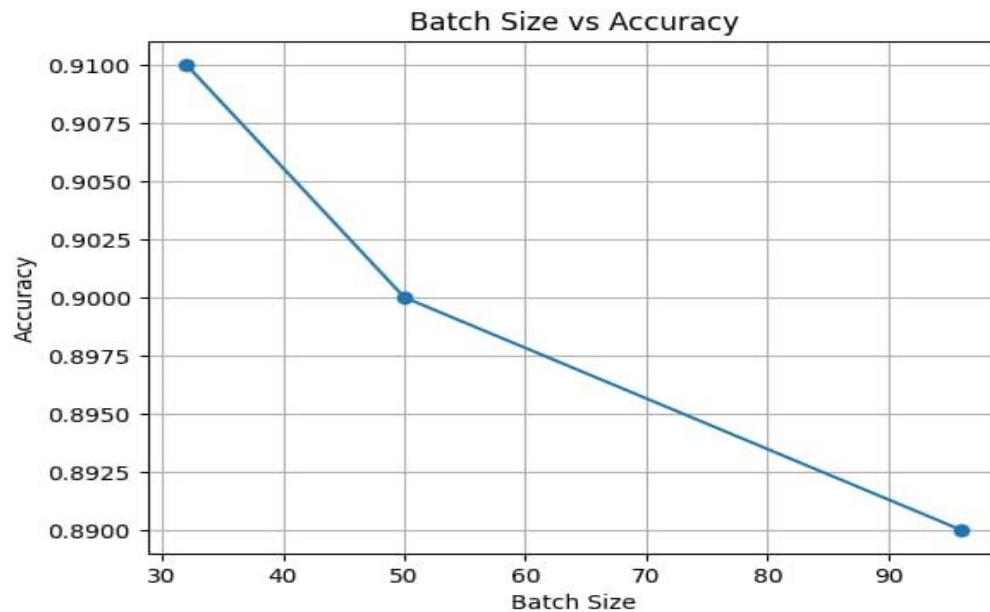
```

```

    model.fit(

```

```
x_train,  
y_train,  
epochs=4,  
batch_size=b,  
validation_split=0.25,  
verbose=1  
)  
  
loss,acc = model.evaluate(x_test,y_test)  
  
batch_acc.append(acc)  
  
plt.figure()  
  
plt.plot(batch_list,batch_acc,marker='o')  
  
plt.title("Batch Size vs Accuracy")  
plt.xlabel("Batch Size")  
plt.ylabel("Accuracy")  
  
plt.show()  
Output:
```



EXPERIMENT 4 : Effect of Optimizer

```
optimizers = ['adam','sgd','rmsprop']  
opt_acc = []
```

```
for o in optimizers:
```

```
    print("Optimizer:",o)
```

```
    model = build_model(opt=o)
```

```
    model.fit(  
        x_train,  
        y_train,  
        epochs=4,  
        batch_size=50,  
        validation_split=0.25,  
        verbose=1  
    )
```

```
    loss,acc =  
model.evaluate(x_test,y_test)
```

```
    opt_acc.append(acc)
```

```
plt.figure()
```

```
plt.bar(optimizers,opt_acc)
```

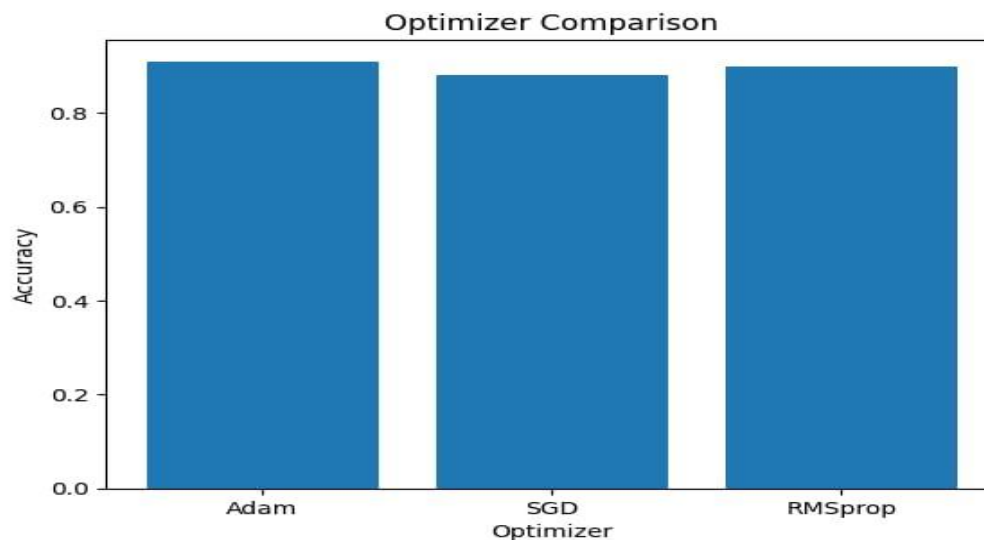
```
plt.title("Optimizer Comparison")
```

```
plt.xlabel("Optimizer")
```

```
plt.ylabel("Accuracy")
```

```
plt.show()
```

Output :-



MY COMMENTS :-

From this experiment, I learned how a Convolutional Neural Network (CNN) can be implemented for image classification using the TensorFlow and Keras libraries. The experiment helped me understand the complete workflow including data preprocessing, building CNN layers, training the model, and evaluating its performance. It also highlighted the importance of selecting suitable hyperparameters to improve model accuracy and learning efficiency.

Learning from Variation Experiments

1. Effect of Kernel Size

Different kernel sizes such as 3×3 , 4×4 , and 6×6 were tested in the convolution layer. It was observed that smaller kernels generally produced better results because they capture detailed features from the images while keeping the number of parameters manageable.

2. Effect of Regularization

L1 regularization was applied to control overfitting during training. The results showed that adding regularization helps the model generalize better by limiting very large weight values and improving performance on unseen data.

3. Effect of Batch Size

Various batch sizes including 32, 50, and 96 were experimented with. Smaller batch sizes improved learning stability, whereas larger batch sizes helped reduce training time but sometimes slightly affected the final accuracy.

4. Effect of Optimization Algorithm

Different optimizers such as Adam, SGD, and RMSprop were compared. Among them, Adam optimizer showed faster convergence and produced comparatively better accuracy for this dataset.